Python Data Science Cheatsheet

CMPT 353: Computational Data Science (Exercises e1 - e4)

Computational Data Science Cheatsheet

June 18, 2026

Contents

1. NumPy: High-Performance Numerical Computing

Array Loading & Saving

```
import numpy as np

# Load a compressed .npz archive containing multiple arrays
data = np.load('monthdata.npz')
totals = data['totals']  # Extract array by key name
counts = data['counts']

# Save multiple arrays to a compressed .npz file
np.savez('monthdata.npz', totals=totals_values, counts=counts_values)
```

Dimensional Reductions & Axis Operations

Operations along `axis=0` are column-wise (reducing rows), while `axis=1` are row-wise (reducing columns).

```
# Calculate total precipitation per city (row-wise summation)
row_totals = np.sum(totals, axis=1)

# Find the row index (city) with the lowest total precipitation
lowest_city_idx = np.argmin(row_totals)

# Calculate column-wise average precipitation (handling division by counts)
col_averages = np.sum(totals, axis=0) / np.sum(counts, axis=0)
```

Advanced Reshaping & Restructuring

Reshaping allows rearranging dimensions to compute multi-step or multi-period aggregations without explicit loops.

```
# Grouping monthly data into quarterly chunks
# Suppose totals has shape (cities, 12 months)
cities_count, months_count = totals.shape

# 1. Reshape the 2D array into a temporary 2D array where monthly segments are grouped in
↪  3s
# Total elements = (cities_count * months_count). Divide by 3 to get total 3-month bins.
temp_quarters = totals.reshape(int(cities_count * months_count / 3), 3)

# 2. Sum along the second axis (each quarter row chunk of 3 months)
quarter_sums = temp_quarters.sum(axis=1)

# 3. Reshape back into a 2D format: (cities_count, quarters_count)
quarterly_totals = quarter_sums.reshape(cities_count, int(months_count / 3))
```

Trigonometric & Math Operations

Many scientific formulas require element-wise mathematical calculations using NumPy's vectorized functions.

```
# Degree-to-Radian constant conversion
P = np.pi / 180

# Vectorized functions
cos_vals = np.cos(arr)
sin_vals = np.sin(arr)
sqrt_vals = np.sqrt(arr)
```

```
arcsin_vals = np.arcsin(arr)
```

---

2. Pandas: Data Analysis & Manipulation

File Input/Output (I/O)

```
import pandas as pd

# Read CSV, parsing specific column as datetime, and set primary index
df_csv = pd.read_csv('precipitation.csv', parse_dates=['date']).set_index('name')

# Read Compressed JSON (NDJSON format, lines=True) with datetime conversion
df_json = pd.read_json('stations.json.gz', lines=True, convert_dates=['timestamp'])

# Save DataFrame to CSV (keeping the index)
df_csv.to_csv('totals.csv', index=True)
```

Data Cleansing & Index Management

```
# Remove rows with null values in specific columns
cleaned_df = df.dropna(subset=['population', 'area'])

# Filter rows matching comparison conditions
filtered_df = cleaned_df[cleaned_df['area'] <= 10000]

# Retrieve row-index label (ID) representing extreme values (e.g. minimum)
lowest_city_name = totals_df.sum(axis=1).idxmin()

# Manage index representation
df_reset = df.reset_index().rename(columns={'old_name': 'new_name'})
df_reset.index.name = 'station_id'
df_reset.columns.name = 'month'
```

Custom Row/Element-Wise Functions (`apply`)

```
# Convert pandas timestamp Series to a custom formatted string representation
def date_to_month(d):
    return '%04i-%02i' % (d.year, d.month)

df['month'] = df['date'].apply(date_to_month)

# Apply a custom function row-by-row (axis=1) passing an external DataFrame as arguments
df['avg_tmax'] = df.apply(find_closest_station_temp, stations=stations_df, axis=1)
```

Joining DataFrames

```
# Perform inner join on indices to align different datasets
combined_df = df_gps.join(df_accl, how='inner').join(df_phone, how='inner')
```

---

3. Advanced Reshaping & Alignment

The Groupby-Aggregate-Pivot Pattern

Reshaping narrow datasets (one record per observation) into a wide matrix (stations as rows, monthly sums/counts as columns).

```
# 1. Group by key dimensions, run target aggregation, reset index to regain normal
 ↪  columns,
#    and pivot into the desired wide-form layout
monthly_totals = data.groupby(by=["name", "month"]) \
                     .aggregate({'precipitation': 'sum'}) \
                     .reset_index() \
                     .pivot(index="name", columns="month", values="precipitation")

monthly_counts = data.groupby(by=["name", "month"]) \
                     .aggregate({'precipitation': 'count'}) \
                     .reset_index() \
                     .pivot(index="name", columns="month", values="precipitation")
```

Manual Reshaping Loop (Alternative Approach)

For scenarios where indexing must be built procedurally or memory optimization is required:

```
# Extract and sort unique indices
stations = sorted(list(set(data['name'])))
station_to_row = {s: i for i, s in enumerate(stations)}

months = sorted(list(set(data['month'])))
month_to_col = {m: i for i, m in enumerate(months)}

# Pre-allocate container numpy arrays
precip_total = np.zeros((len(stations), len(months)), dtype=np.uint)

# Interate through raw records and populate
for _, row in data.iterrows():
    r = station_to_row[row['name']]
    c = month_to_col[row['month']]
    precip_total[r, c] += row['precipitation']

# Re-build structured DataFrame
totals_df = pd.DataFrame(data=precip_total, index=stations, columns=months)
```

---

4. Datetime Manipulation & Alignment

Datetime Offsets & Parsing

```
# Convert to timezone-aware UTC datetime Series
df['datetime'] = pd.to_datetime(df['time_string'], utc=True)

# Generate minimum baseline time
first_time = df['datetime'].min()

# Add a float seconds offset to a baseline timestamp using pd.to_timedelta
df['timestamp'] = first_time + pd.to_timedelta(df['seconds'] + offset_seconds, unit='s')
```

Temporal Resampling / Rounding

Aligning multiple sensor datasets collected at different frequencies by binning times.

```
# Round timestamp values to a fixed block interval (e.g., 4 seconds)
df['rounded'] = df['timestamp'].dt.round('4s')

# Group by the rounded bins and take the mean of all numeric features
aligned_df = df.groupby('rounded').mean(numeric_only=True)
```

---

5. Data Visualization (Matplotlib)

Multiple Subplots & Figure Sizing

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))

# Subplot 1 (1 row, 2 columns, position 1)
plt.subplot(1, 2, 1)
plt.title("Popularity Distribution")
plt.xlabel("Rank")
plt.ylabel("Views")
plt.plot(sorted_views_array, 'b-')  # Blue solid line

# Subplot 2 (1 row, 2 columns, position 2)
plt.subplot(1, 2, 2)
plt.title("Hourly Correlation")
plt.xlabel("Hour 1 views")
plt.ylabel("Hour 2 views")

# Apply logarithmic scales
plt.xscale("log")
plt.yscale("log")
plt.scatter(df.x, df.y, alpha=0.5, c='red')  # Red scatter points
plt.legend(['Page Views'])

# Export the figure to file (supports PNG, SVG, PDF)
plt.savefig('wikipedia.png', dpi=300)
```

---

6. Computational & Domain Algorithms

Haversine Formula (GPS Distance Over Track)

Calculates the total path distance in meters traversed by consecutive points in a DataFrame.

```
def calculate_path_distance(df):
    P = np.pi / 180
    lat1, lon1 = df['lat'], df['lon']

    # Shift coordinate series down by 1 row to get previous coordinates
    lat2, lon2 = lat1.shift(), lon1.shift()
```

```
    # Haversine distance formula (vectorized)
    a = 0.5 - np.cos((lat2 - lat1) * P)/2 + np.cos(lat1 * P) * np.cos(lat2 * P) * (1 -
↪   np.cos((lon2 - lon1) * P))/2

    # Earth diameter is 12742 km. Multiply by 1000 for total distance in meters.
    total_meters = 12742 * np.arcsin(np.sqrt(a)).sum() * 1000
    return total_meters
```

Haversine Formula (Point-to-Array Correlation)

Finds the distance from a single coordinate (e.g. city) to a collection of coordinates (e.g. weather stations).

```
def haversine_point_to_array(city, stations):
    lat1 = np.radians(city['latitude'])
    lon1 = np.radians(city['longitude'])
    lat2 = np.radians(stations['latitude'])
    lon2 = np.radians(stations['longitude'])

    dlat = lat2 - lat1
    dlon = lon2 - lon1

    a = np.sin(dlat / 2.0)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2.0)**2
    c = 2 * np.arcsin(np.sqrt(a))

    r = 6371  # Earth radius in km
    return r * c

# Applied across all cities to find the closest weather station's temperature reading
def get_best_tmax(city_row, stations_df):
    distances = haversine_point_to_array(city_row, stations_df)
    # Find position of minimum distance Weather Station
    best_idx = np.argmin(distances.values)
    return stations_df.iloc[best_idx]['avg_tmax']

# Usage:
cities['avg_tmax'] = cities.apply(get_best_tmax, stations_df=stations, axis=1)
```

Signal Optimization via Cross-Correlation

Finds the optimal time offset to sync asynchronous sensors (e.g. accelerometer and phone gyroscope).

```
best_offset = 0
max_corr = -np.inf

# Grid search time offsets from -5 to +5 seconds in 0.1-second increments
for offset in np.linspace(-5.0, 5.0, 101):
    # Apply offset to sensor times
    phone['timestamp'] = first_time + pd.to_timedelta(phone['time'] + offset, unit='s')
    phone['rounded'] = phone['timestamp'].dt.round('4s')
    phone_grouped = phone.groupby('rounded').mean(numeric_only=True)

    # Compute cross-correlation sum between aligned sensor dimensions
    corr = (phone_grouped['gFx'] * accl_grouped['x']).sum()

    if corr > max_corr:
        max_corr = corr
        best_offset = offset
```

7. XML, GPX and DOM Processing

Parsing GPX/XML with Namespaces

```
import xml.etree.ElementTree as ET

def get_gpx_points(input_gpx):
    tree = ET.parse(input_gpx)
    root = tree.getroot()

    # Define XML GPX namespace mapping
    namespace = {'gpx': 'http://www.topografix.com/GPX/1/0'}

    data = []
    # Query elements matching GPX track point tags
    for trkpt in root.findall('.//gpx:trkpt', namespace):
        lat = float(trkpt.get('lat'))
        lon = float(trkpt.get('lon'))
        time_text = trkpt.find('gpx:time', namespace).text

        data.append({
            'datetime': pd.to_datetime(time_text, utc=True),
            'lat': lat,
            'lon': lon
        })

    return pd.DataFrame(data)
```

Generating & Saving GPX Files with minidom

```
from xml.dom.minidom import getDOMImplementation

def output_pretty_gpx(points, output_filename):
    xmlns = 'http://www.topografix.com/GPX/1/0'
    impl = getDOMImplementation()

    # Initialize DOM structure
    doc = impl.createDocument(None, 'gpx', None)
    trk = doc.createElement('trk')
    doc.documentElement.appendChild(trk)
    trkseg = doc.createElement('trkseg')
    trk.appendChild(trkseg)

    # Procedurally append points
    for _, pt in points.iterrows():
        trkpt = doc.createElement('trkpt')
        trkpt.setAttribute('lat', '%.10f' % (pt['lat']))
        trkpt.setAttribute('lon', '%.10f' % (pt['lon']))

        time_node = doc.createElement('time')

↪   time_node.appendChild(doc.createTextNode(pt['datetime'].strftime("%Y-%m-%dT%H:%M:%SZ")))
```

```
        trkpt.appendChild(time_node)
        trkseg.appendChild(trkpt)

    # Set outer namespace
    doc.documentElement.setAttribute('xmlns', xmlns)

    # Write formatted pretty XML to file
    with open(output_filename, 'w') as fh:
        fh.write(doc.toprettyxml(indent='  '))
```

---

8. Data Smoothing and Filtering

LOESS / LOWESS Smoothing (1D Local Regression)

```
import statsmodels.api as sm

# Extract lowess nonparametric regression function
lowess = sm.nonparametric.lowess

# Smooth temperature readings by timestamp (frac controls sliding window scale)
loess_smoothed = lowess(cpu_data['temperature'], cpu_data['timestamp'], frac=0.05)

# Returns a numpy array with shape (N, 2), where:
# Column 0: original X timestamps, Column 1: smoothed Y temperatures
plt.plot(cpu_data['timestamp'], loess_smoothed[:, 1], 'r-', label='LOESS')
```

Kalman Filtering (pykalman)

Scenario A: Multi-Dimensional GPS Track Smoothing Uses noisy GPS coordinates and multi-dimensional magnetic fields (`Bx`, `By`) to reconstruct true smooth paths.

```
from pykalman import KalmanFilter

# 4D state-space vector: [latitude, longitude, magnetic_Bx, magnetic_By]
kf = KalmanFilter(
    initial_state_mean=p.iloc[0][['lat', 'lon', 'Bx', 'By']],
    observation_matrices=np.eye(4),  # Identity observation matrix (maps state 1-to-1)
    observation_covariance=np.diag([(5/1e5)**2, (5/1e5)**2, 2, 2]),  # Observational
 ↪  noise variance
    transition_matrices=[
        [1, 0, 0, 1e-7],
        [0, 1, 1e-7, 0],
        [0, 0, 1,    0],
        [0, 0, 0,    1]
    ],
    transition_covariance=np.diag([(1/1e5)**2, (1/1e5)**2, 2, 2]) # State transition
 ↪  uncertainty
)

# Smooth GPS trace observations
smoothed_results, _ = kf.smooth(p[['lat', 'lon', 'Bx', 'By']])
smoothed_points = pd.DataFrame(smoothed_results, columns=['lat', 'lon', 'Bx', 'By'],
 ↪  index=p.index)
```

Scenario B: System Estimation with Linear Feature Dependencies Models temperature fluctuations linked to physical dependencies, such as CPU utilization, system load, and fan speed.

```
# Initial state of features: [temperature, cpu_percent, sys_load_1, fan_rpm]
initial_state = kalman_data.iloc[0]

# Transition matrix models how each feature impacts other features in the next state
 ↪  timestep
transition = [
    [0.99, 0.5, 0.2, -0.001],  # Temp depends heavily on previous temp, cpu, load, and
 ↪  inverse fan RPM
    [0.1,  0.4, 2.1,  0.0  ],  # CPU usage dependencies
    [0.0,  0.0, 0.95, 0.0  ],  # System load dependencies
    [0.0,  0.0, 0.0,  1.0  ]   # Fan speed dependencies
]

kf = KalmanFilter(
    transition_matrices=transition,
    transition_covariance=np.diag([0.5, 0.5, 0.5, 0.5]) ** 2,
    observation_covariance=np.diag([0.5, 0.5, 0.5, 0.5]) ** 2,
    initial_state_mean=initial_state
)

kalman_smoothed, _ = kf.smooth(kalman_data)

# Extract column 0 (smoothed temperature column)
plt.plot(cpu_data['timestamp'], kalman_smoothed[:, 0], 'g-', label='Kalman')
```